SIMATS ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining

Assignment 1 - CO1

CO1 Coverage: *Operational vs decision-support systems, data warehouse architecture, ETL, dimensional data modelling (star and snowflake schemas), data cubes, OLAP operations, query processing, open-source BI tools and KNIME for reporting.*

Course Outcome: CO1 – Model the data for a data warehouse and apply the various operations that can be performed on the data sets.

Assignment Title: Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain (Total Marks - 50)

Duration: 1 week

Scenario — FreshCart Retail Analytics

FreshCart operates 120 supermarkets across six states. Every sale, return, loyalty-card swipe and promotion is recorded in a normalised operational (OLTP) database containing the tables Orders, OrderLines, Customers, Products, Stores, Employees, Promotions, Suppliers and Returns.

Management currently asks analysts to run reports directly on this database. Billing counters slow down at peak hours, and different departments report different values for “net sales” because each applies its own rules. Management has asked you to design a **data warehouse** that supports questions such as:

- Which product categories show declining net sales, quarter over quarter, in each region?
- Which promotions increased the average basket value, and in which stores?
- How does weekend sales performance differ from weekday performance for each department?
- Which customer segments return the most items, and of which products?
- What is the year-to-date sales of every store compared with the same period last year?

The sales fact data may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ sales line items

You are required to investigate, design and compare alternative warehouse designs, and to demonstrate OLAP operations on the design you recommend. Your solution must remain practical as data grows. Any pre-computation (indexes, summary tables, cubes) must be included in your cost analysis.

Design Challenge:

FreshCart now launches an online channel that sends about 20,000 orders per day from a new source system. Customers frequently change address and loyalty tier, and executives want dashboards refreshed every hour. Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – ANALYSE THE DECISION-SUPPORT REQUIREMENT:

1.1 Need for a Data Warehouse:

FreshCart uses an OLTP system for daily transactions. However, it is not suitable for complex analysis because:

1. Complex queries can slow down billing operations.
2. Historical data analysis is difficult.
3. Different departments may produce inconsistent reports.
4. It does not efficiently support management decisions.

A Data Warehouse provides integrated historical data for analysis and decision-making.

1.2 Business Questions:

Business Question	Measures	Dimensions
Which products have declining sales?	Net Sales	Product, Time, Store
Which promotions increase sales?	Sales, Discount	Promotion, Date
Which stores perform best?	Net Sales	Store, Region
Which customers return more products?	Return Quantity	Customer, Product
What are yearly sales?	Total Sales	Year, Store
Which products are most profitable?	Profit	Product, Category
Which regions generate more revenue?	Net Sales	Region, Time
Which products have low sales?	Quantity Sold	Product, Store

1.3 OLTP vs DSS:

OLTP	DSS
Handles daily transactions	Supports decisions
Current data	Historical data
Simple queries	Complex queries
Operational users	Managers and analysts
Frequent updates	Periodic loading

Conclusion:

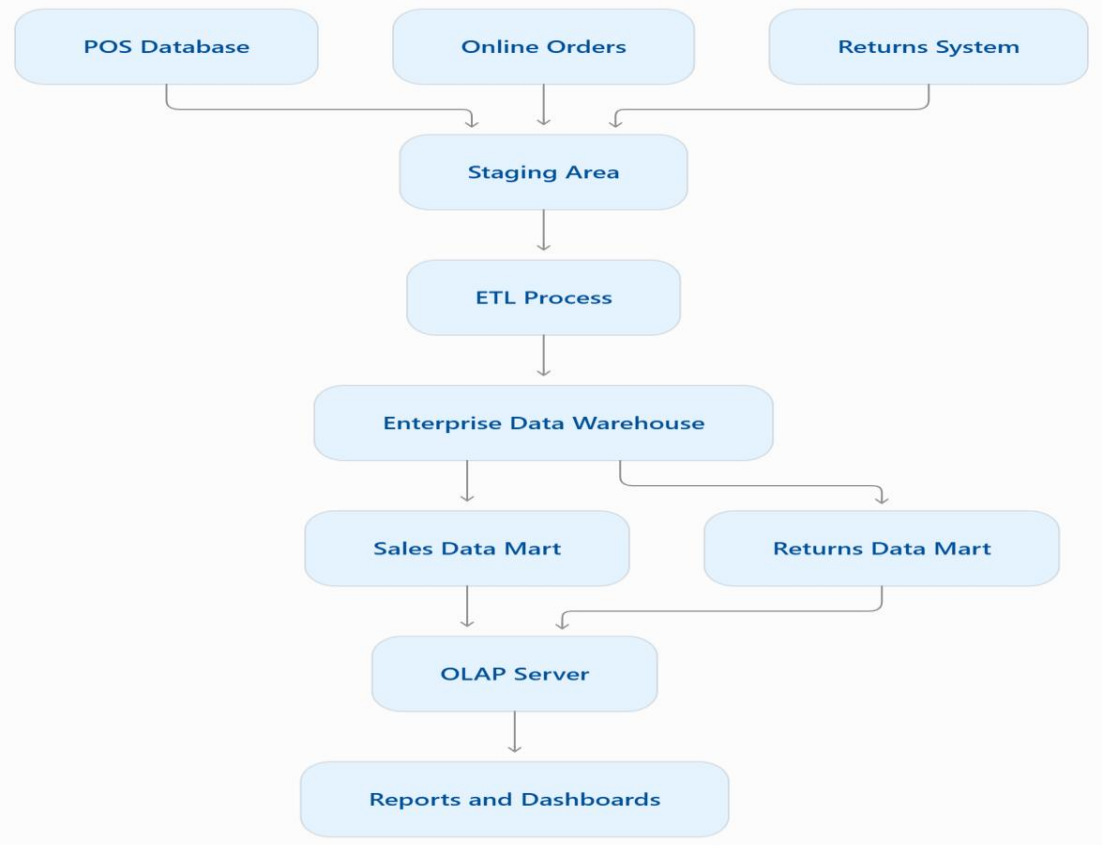
For FreshCart, analytical questions such as year-to-date sales, customer return patterns and regional sales comparisons cannot be efficiently answered using the OLTP database. A separate DSS provides historical information, consistent calculations and better analytical performance.

TASK 2 – DATA WAREHOUSE ARCHITECTURE:

2.1 Proposed Architecture:

FreshCart should use an Enterprise Data Warehouse with dependent data marts.

2.2 Architecture Diagram:



2.3 Components:

1. Data Sources: Collect sales, customer, product, store, promotion and return information from operational databases.
2. Staging Area: Temporarily stores extracted data before transformation.
3. ETL Process: Extracts, cleans, transforms and loads the data into the warehouse.
4. Data Warehouse: Stores integrated historical information.
5. Metadata Repository: Stores information about data definitions, source mappings and transformation rules.
6. Data Marts: Provide department-specific information for sales and returns analysis.
7. OLAP Server: Performs analytical operations such as roll-up, drill-down, slice, dice and pivot.

8. Front-End Tools: Metabase, Apache Superset and KNIME generate reports and dashboards.

2.4 Justification:

An Enterprise Data Warehouse is suitable because FreshCart operates 120 supermarkets across six states. It provides a centralised source of reliable information.

Dependent data marts ensure that all departments use the same business rules and sales calculations.

TASK 3 – DESIGN THE STAR SCHEMA

3.1 Business Process:

The selected business process is Retail Sales Analysis.

The warehouse stores information about sales transactions, products, customers, stores, dates and promotions.

3.2 Grain of the Fact Table

The grain of the FactSales table is:

One row represents one product line within a single customer order at a particular store on a specific date.

This grain ensures that every sales transaction is recorded at the individual product-line level.

3.3 Fact Table

The main fact table is FactSales.

Measure	Description	Classification
Quantity	Number of products sold	Additive
GrossSales	Total sales before discounts	Additive
DiscountAmount	Total discount applied	Additive
NetSales	Sales after discounts and applicable returns	Additive
UnitPrice	Price per unit	Non-additive
ProfitMargin	Profit percentage	Non-additive

Formula:

Gross Sales = Quantity × Unit Price

Net Sales = Gross Sales − Discount Amount − Allocated Sales Returns

3.4 Dimension Tables:

1. DimDate

- DateKey
- FullDate
- Day
- Month
- MonthName
- Quarter
- Year
- IsWeekend

Hierarchy: Day → Month → Quarter → Year.

2. DimProduct

- ProductKey
- ProductCode
- ProductName
- Brand
- Category
- Department
- PackSize
- SupplierCode

Hierarchy: Product → Category → Department.

3. DimStore

- StoreKey
- StoreCode
- StoreName

- City
- State
- Region
- StoreType

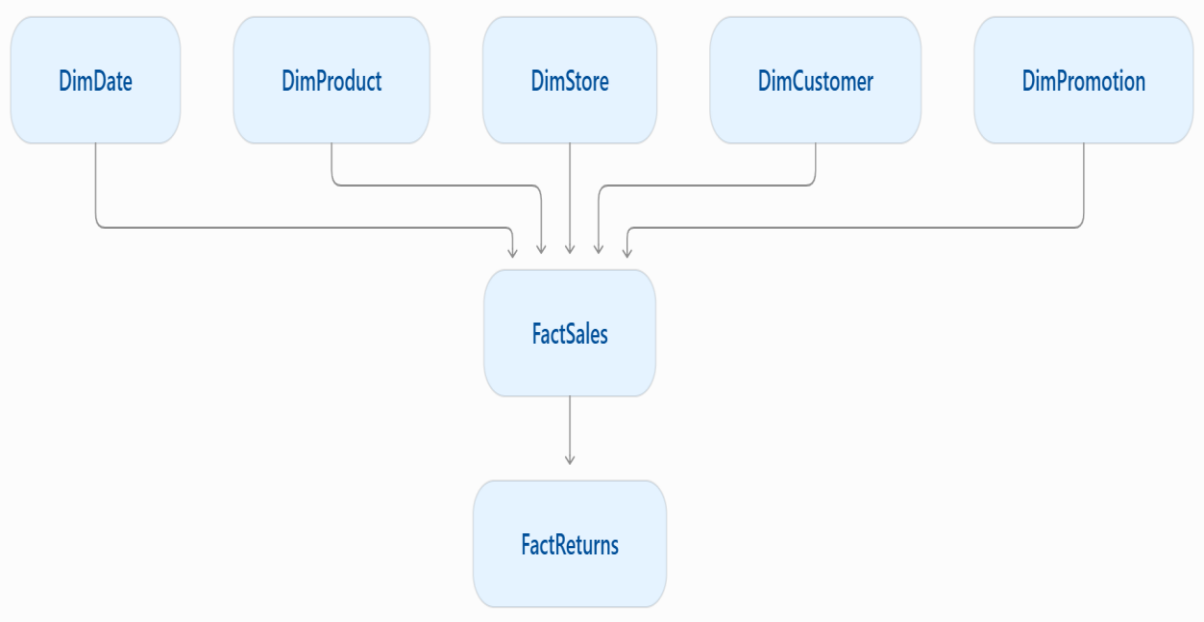
Hierarchy: Store → City → State → Region.

4. DimCustomer

- CustomerKey
- CustomerID
- CustomerName
- Segment
- Address
- City
- State
- LoyaltyTier

Hierarchy: Customer → Segment.

3.5 Star Schema Diagram:



FactSales contains foreign keys referencing the five dimension tables. FactReturns is a separate fact table that can share conformed dimensions such as Date, Product, Store and Customer.

3.6 SQL DDL Statements:

The following SQL code creates the star schema using MySQL.

```
CREATE DATABASE FreshCartDW;

USE FreshCartDW;


CREATE TABLE DimDate (
 DateKey INT PRIMARY KEY,
 FullDate DATE,
 MonthName VARCHAR(20),
 QuarterNum INT,
 YearNum INT
);


CREATE TABLE DimProduct (
 ProductKey INT PRIMARY KEY,
 ProductName VARCHAR(100),
 Category VARCHAR(50),
 Brand VARCHAR(50)
);


CREATE TABLE DimStore (
 StoreKey INT PRIMARY KEY,
 StoreName VARCHAR(100),
 City VARCHAR(50),
 Region VARCHAR(50)
);
```

```
CREATE TABLE DimCustomer (
 CustomerKey INT PRIMARY KEY,
 CustomerName VARCHAR(100),
 Segment VARCHAR(40)
);

CREATE TABLE DimPromotion (
 PromotionKey INT PRIMARY KEY,
 PromotionName VARCHAR(100),
 PromotionType VARCHAR(50)
);

CREATE TABLE FactSales (
 SalesKey INT PRIMARY KEY,
 DateKey INT,
 ProductKey INT,
 StoreKey INT,
 CustomerKey INT,
 PromotionKey INT,
 Quantity INT,
 GrossSales DECIMAL(12,2),
 DiscountAmount DECIMAL(12,2),
 NetSales DECIMAL(12,2),

 FOREIGN KEY (DateKey) REFERENCES DimDate(DateKey),
 FOREIGN KEY (ProductKey) REFERENCES DimProduct(ProductKey),
 FOREIGN KEY (StoreKey) REFERENCES DimStore(StoreKey),
 FOREIGN KEY (CustomerKey) REFERENCES DimCustomer(CustomerKey),
```

FOREIGN KEY (PromotionKey) REFERENCES DimPromotion(PromotionKey)

);

Conclusion:

Star schema simplifies reporting and improves analytical query performance.

TASK 4 – STAR SCHEMA VS SNOWFLAKE SCHEMA:

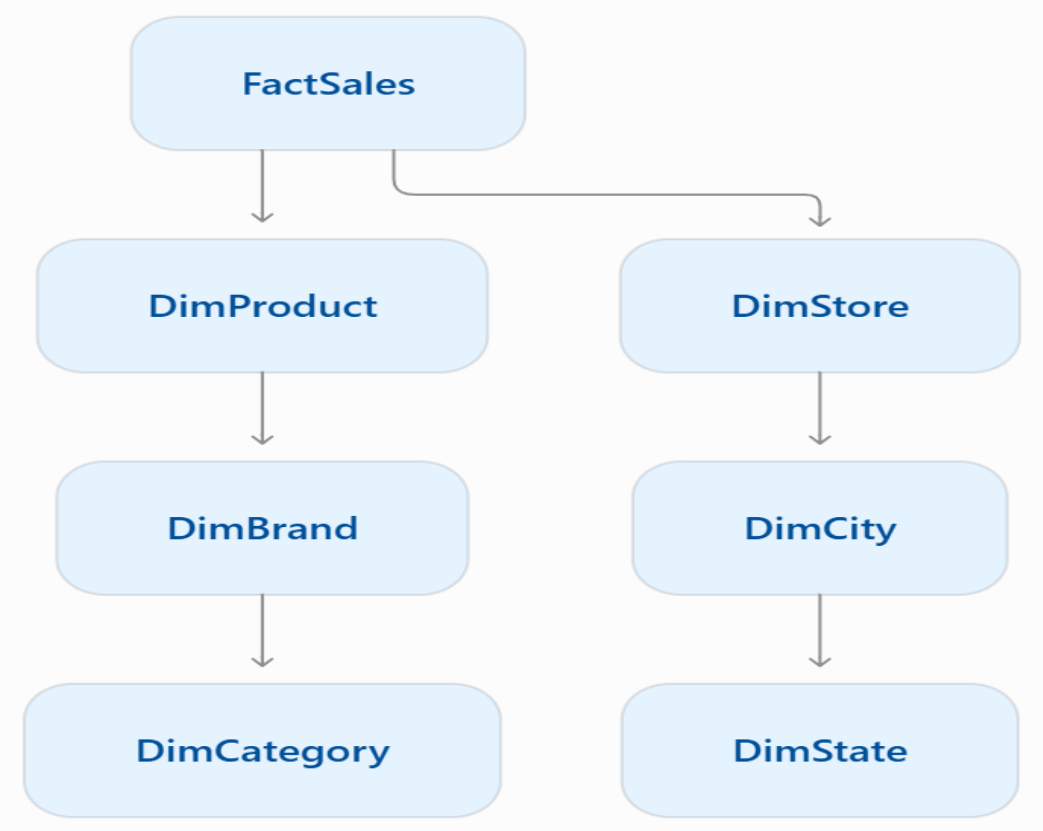
4.1 Snowflake Schema:

A snowflake schema divides dimension tables into smaller related tables to reduce repeated information.

Example:

- Product → Brand → Category → Department.
- Store → City → State → Region.

4.2 Diagram:



4.3 Comparison:

Star Schema	Snowflake Schema
Simple structure	Complex structure
Fewer joins	More joins
Faster reporting in many cases	Additional join processing
More repeated data	Less repeated data
Easy to understand	Harder to maintain

Conclusion:

FreshCart should use a Star Schema because it supports faster and simpler sales reporting.

TASK 5 – ETL PROCESS:

5.1 Introduction:

ETL means Extract, Transform and Load. It transfers data from operational systems into the Data Warehouse.

5.2 ETL Diagram:



5.3 Surrogate Keys:

A surrogate key is a unique artificial identifier generated by the data warehouse for each record in a dimension table. It helps identify records and maintain relationships between fact and dimension tables. For example, ProductKey 101 represents Basmati Rice, 102 represents Olive Oil, and 103 represents Tea.

5.4 Slowly Changing Dimensions (SCD):

Slowly Changing Dimension (SCD) Type 2 is used to maintain historical changes in dimension data. Instead of replacing old information, it creates a new record whenever a customer's details change. For example, if a customer moves from Chennai to Coimbatore, the old address is retained and a new record is created with the updated address.

5.5 Conclusion:

The ETL process improves data quality by cleaning and transforming data before loading it into the warehouse. It also maintains historical information and ensures reliable data for business analysis.

TASK 6 – DATA CUBE AND OLAP OPERATIONS:

6.1 Data Cube:

A Data Cube represents sales information across multiple dimensions.

FreshCart uses:

1. Time
2. Product
3. Store
4. Customer Segment

Measure: Net Sales.

6.2 Number of Cuboids:

With hierarchies:

$$5 \times 4 \times 4 \times 2 = 160$$

Without hierarchies:

$$2^4 = 16$$

1.Base Cuboid: Detailed sales information.

2.Apex Cuboid: Total sales across all dimensions.

6.3 OLAP Operations:

1. Roll-up: Summarises daily sales into monthly sales.

```
SELECT d.MonthName, SUM(f.NetSales)
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
GROUP BY d.MonthName;
```

2. Drill-down: Displays detailed sales information.

```
SELECT d.FullDate, SUM(f.NetSales)
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
GROUP BY d.FullDate;
```

3. Slice: Selects one dimension value:

```
SELECT SUM(f.NetSales)
FROM FactSales f
JOIN DimDate d ON f.DateKey = d.DateKey
WHERE d.YearNum = 2026;
```

4. Dice: Selects multiple dimension values:

```
SELECT p.Category, s.Region, SUM(f.NetSales)
FROM FactSales f
JOIN DimProduct p ON f.ProductKey = p.ProductKey
JOIN DimStore s ON f.StoreKey = s.StoreKey
WHERE p.Category = 'Groceries'
AND s.Region = 'South'
GROUP BY p.Category, s.Region;
```

5. Pivot:

Changes the presentation of rows and columns to compare sales across regions.

Conclusion:

OLAP helps FreshCart analyse sales from different perspectives.

TASK 7 – DEVELOP AN OLAP APPLICATION:

7.1 Introduction:

An OLAP application allows users to analyse sales using Roll-up, Drill-down, Slice, Dice and Pivot operations.

7.2 Install Libraries:

pip install pandas streamlit

7.3 Dataset Generation Code:

Create generate_data.py.

```
import pandas as pd
import random

data = []

for i in range(1000):
    qty = random.randint(1, 10)
    price = random.randint(50, 1000)

    data.append({
        "category": random.choice(
            ["Groceries", "Beverages", "Snacks"]
        ),
        "store": random.choice(
```

```
            ["Chennai", "Pune", "Mumbai"]
        ),
        "quantity": qty,
        "net_sales": qty * price
    })

pd.DataFrame(data).to_csv("sales.csv", index=False)

print("Dataset created successfully")
```

7.4 OLAP Application Code:

Create olap_app.py.

```
import pandas as pd
import streamlit as st

st.title("FreshCart OLAP Dashboard")

df = pd.read_csv("sales.csv")

operation = st.selectbox(
    "Operation",
    ["Roll-up", "Drill-down", "Slice", "Dice", "Pivot"]
)

measure = st.selectbox(
    "Measure",
    ["net_sales", "quantity"]
)

if operation == "Slice":
    category = st.selectbox(
```

```
        "Category", df["category"].unique()
    )
    df = df[df["category"] == category]

elif operation == "Dice":
    store = st.selectbox(
        "Store", df["store"].unique()
    )
    df = df[df["store"] == store]

if operation == "Pivot":
    result = df.pivot_table(
        index="category",
        columns="store",
        values=measure,
        aggfunc="sum",
        fill_value=0
    )
else:
    result = df.groupby("category")[measure].sum()

st.write(result)
st.bar_chart(result)

st.metric("Total", round(df[measure].sum(), 2))
```

7.5 Execution:

python generate_data.py

streamlit run olap_app.py

Conclusion:

The application displays sales results using different OLAP operations.

TASK 8 – QUERY PROCESSING PERFORMANCE:

8.1 Introduction:

Query performance measures how quickly a database executes analytical queries.

FreshCart compares:

1. Normalised database.
2. Star schema.
3. Star schema with indexes and aggregate tables.

8.2 Sample Query:

```
SELECT
p.Category,
SUM(f.NetSales) AS TotalSales
FROM FactSales f
JOIN DimProduct p
ON f.ProductKey = p.ProductKey
GROUP BY p.Category;
```

8.3 Performance Improvement:

The following techniques improve query performance:

1. Create indexes on foreign keys.
2. Use aggregate tables for frequently requested reports.
3. Avoid unnecessary columns in queries.
4. Partition large fact tables by date.
5. Use query execution plans to identify slow operations.

8.4 Index Example:

```
CREATE INDEX idx_sales_date
ON FactSales(DateKey);
```

```
CREATE INDEX idx_sales_product
ON FactSales(ProductKey);

CREATE INDEX idx_sales_store
ON FactSales(StoreKey);
```

8.5 Validation:

Use EXPLAIN to inspect the query execution plan.

```
EXPLAIN
SELECT SUM(NetSales)
FROM FactSales
WHERE DateKey = 20261103;
```

Record execution times for each design using the same dataset and query. Actual performance values must be measured by running the queries.

Conclusion:

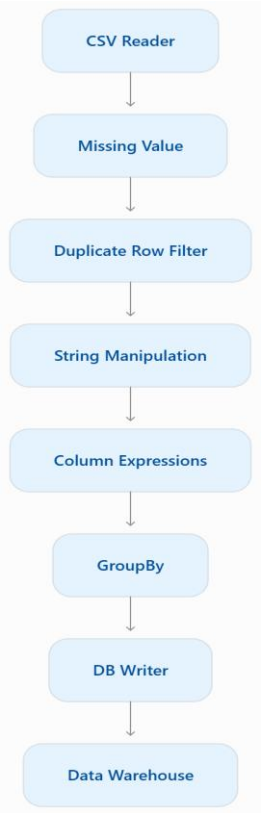
Indexing, aggregation and query optimisation improve FreshCart's reporting performance.

TASK 9 – KNIME WORKFLOW:

9.1 Introduction:

KNIME is a visual analytics platform used for data integration, cleaning and transformation.

9.2 Workflow:



9.3 Nodes Used:

1. CSV Reader: Imports sales data.
2. Missing Value: Handles missing records.
3. Duplicate Row Filter: Removes duplicate records.
4. String Manipulation: Standardises names.
5. GroupBy: Aggregates sales.
6. DB Writer: Loads processed data into the database.

Conclusion:

KNIME simplifies ETL through a visual workflow without requiring extensive programming.

TASK 10 – Final Design Recommendation:

FreshCart should implement an Enterprise Data Warehouse using a Star Schema.

The proposed solution includes:

1. Centralised storage for all retail data.
2. ETL for data cleaning and integration.
3. Star Schema for efficient reporting.
4. OLAP for multidimensional analysis.
5. Streamlit and Metabase for dashboards.
6. Indexing and aggregation for better performance.

Conclusion:

The proposed solution improves data quality, reporting speed and management decision-making.

TASK 11 – DAY-4 DESIGN CHALLENGE:

11.1 Introduction:

FreshCart plans to introduce 200 new stores and requires near-real-time reporting.

The existing warehouse must be enhanced to handle increased data volume and frequent updates.

11.2 Challenges:

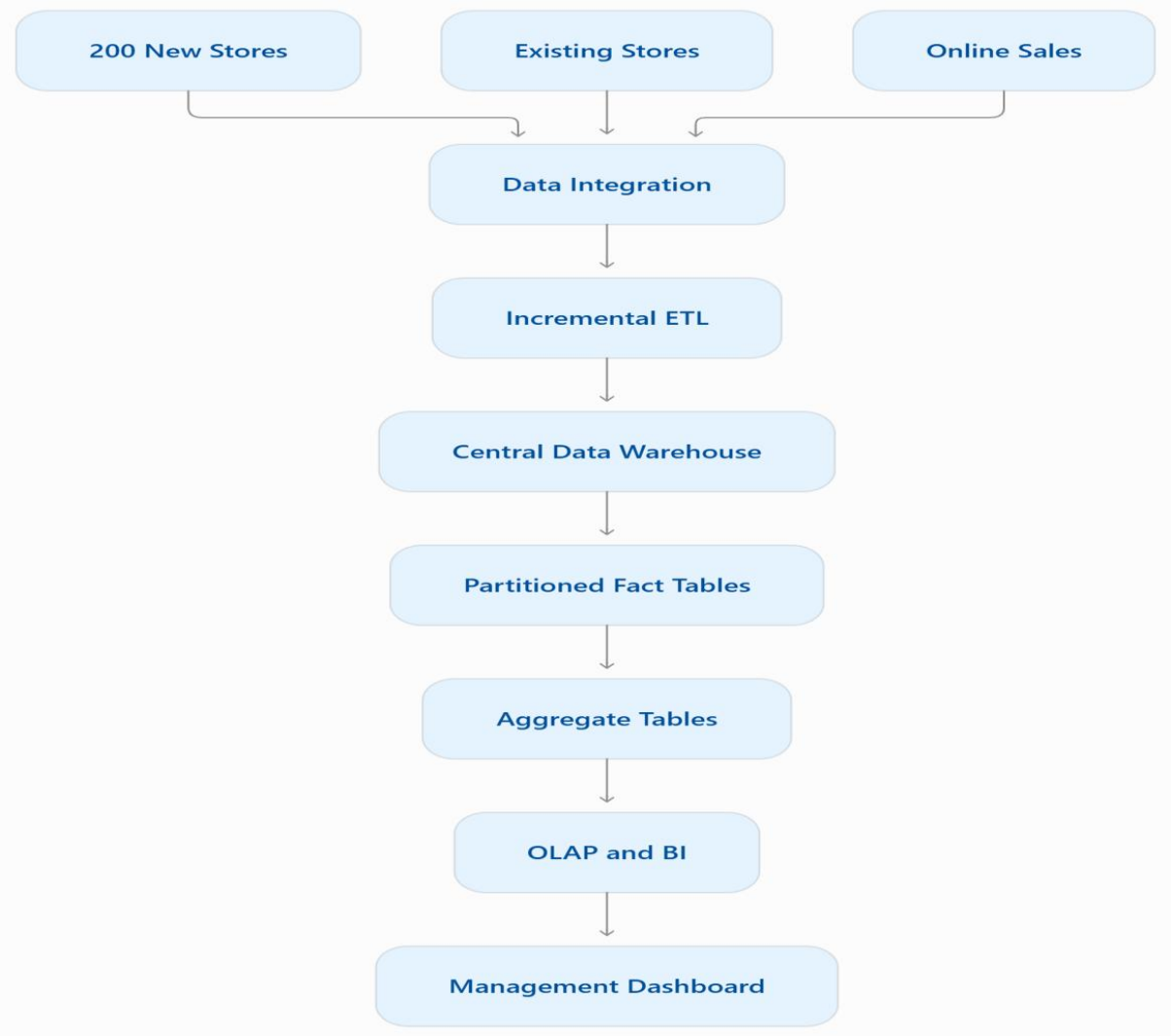
1. Increased transaction volume.
2. More complex ETL processing.
3. Increased storage requirements.
4. Faster reporting requirements.
5. Maintaining historical information.
6. Data consistency across stores.

11.3 Proposed Improvements:

1. Use incremental ETL instead of full loading.
2. Partition fact tables by date.

3. Use SCD Type 2 for historical changes.
4. Introduce aggregate tables for dashboards.
5. Use automated data-quality checks.
6. Schedule frequent data loads.

11.4 Updated Architecture:



11.5 Data Volume Estimation:

Assume each store generates 1,000 order lines per day.

For 200 new stores:

$$200 \times 1000 = 200000$$

New order lines per day = 200,000.

For 30 days:

$$200000 \times 30 = 6000000$$

Monthly order lines = 6,000,000.

This is an estimate based on the assumed daily transaction volume**.**

11.6 Expected Benefits:

1. Faster reporting.
2. Better scalability.
3. Improved data accuracy.
4. Efficient historical analysis.
5. Better inventory and sales decisions.

Conclusion:

Incremental ETL, partitioning and aggregate tables help FreshCart manage its growing business efficiently.

OVERALL CONCLUSION:

The proposed FreshCart Data Warehouse uses ETL, Star Schema, OLAP and BI tools to integrate and analyse retail data. It provides reliable historical reports, improves query performance and supports better business decisions**.**